

Day 9 - CSS Advanced: Pseudo-classes, Transitions, Custom Properties

Day 9 — CSS Advanced: Pseudo-classes, Pseudo-elements, Transitions, Animations, Custom Properties

Objectives

- Style elements based on state and position using pseudo-classes
- Insert generated content using pseudo-elements
- Animate smoothly between states with `transition`
- Build a full animation sequence with `@keyframes`
- Define and reuse your own values with CSS custom properties (variables)

Pseudo-classes — styling based on state or position, without extra HTML or JavaScript

A **pseudo-class** selects an element based on some **state** or **position** it's currently in, rather than its tag, class, or id — written with a single colon:

```
a:hover {
  color: red;    /* while the mouse is hovering over a link */
}
input:focus {
  border-color: blue;    /* while an input currently has keyboard focus (e.g. just clicked into) */
}
li:first-child {
  font-weight: bold;    /* only the FIRST li among its siblings */
}
li:last-child {
  border-bottom: none;    /* only the LAST li among its siblings */
}
li:nth-child(2) {
  color: gray;    /* only the SECOND li among its siblings, counting from 1 */
}
tr:nth-child(odd) {
  background-color: #f4f4f4;    /* every odd-numbered row -- a classic "zebra striping" pattern */
}
```

`:hover` and `:focus` are especially important for a reason beyond mere visual polish: they're how you give a visitor **feedback** that something is interactive and currently being pointed at or interacted with — a link that visually changes on hover signals "this is clickable" before the visitor even clicks, and a clearly visible `:focus` style is genuinely important for anyone navigating your page with a keyboard instead of a mouse (you'll revisit this specific accessibility point on Day 10). **Never remove focus styling (`outline: none;`) without**

providing an equally visible replacement — doing so makes a page unusable for keyboard-only visitors, who rely entirely on that visual indicator to know where they currently are on the page.

`:nth-child()` accepts more than just plain numbers: `:nth-child(odd)`/`:nth-child(even)` (alternating rows), `:nth-child(3n)` (every third element), and others — worth knowing these exist, even if you only reach for the simple cases most often.

Pseudo-elements — targeting a *part* of an element, or inserting content that isn't in your HTML at all

A **pseudo-element** is written with a double colon (`::`) and lets you style a specific *part* of an element, or insert generated content that doesn't exist anywhere in your actual HTML:

```
p::first-line {
  font-weight: bold;    /* styles only the first LINE of text, wherever it happens to wrap */
}
.quote::before {
  content: "\201C";      /* inserts an opening curly quote mark BEFORE the element's actual content */
}
.quote::after {
  content: "\201D";      /* inserts a closing curly quote mark AFTER the element's actual content */
}
```

`::before` and `::after` require a `content` property to actually display anything (even `content: ""`; for an empty, purely decorative box) and are extremely common in real CSS for small decorative touches — a bullet icon, a decorative quote mark, a small badge — without needing to add an extra, purely-decorative element into your actual HTML just to hold it.

`transition` — smoothly animating between two states

Without any transition, a CSS property that changes (say, on `:hover`) snaps instantly from its old value to its new one. `transition` tells the browser to smoothly interpolate between the two values over a specified duration instead:

```
.button {
  background-color: steelblue;
  transition: background-color 0.3s ease;
}
.button:hover {
  background-color: darkblue;
}
```

Read the `transition` declaration as: "for the `background-color` property specifically, when it changes, animate smoothly over `0.3s` (300 milliseconds), using an `ease` timing curve (starts slightly slow, speeds up, slows down again near the end — the most natural-feeling default for most UI animations)." You can transition multiple properties at once, either by listing several comma-separated, or using `transition: all 0.3s ease`; to apply the same timing to every property that changes — the latter is convenient but slightly less precise, since it applies to literally every animatable property, whether you intended that or not.

This single technique — hover/focus state change plus a `transition` — is responsible for a huge share of what makes a modern website feel "polished" versus feeling static and abrupt; it's a small addition with a

disproportionately large effect on how professional an interface feels.

`@keyframes` — defining a full, multi-step animation sequence

`transition` only animates between exactly two states (the "before" and "after"). For a more elaborate sequence — several steps, or something that repeats automatically without needing a trigger like `:hover` — use `@keyframes`:

```
@keyframes bounce {
  0% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
  100% { transform: translateY(0); }
}

.bouncing-ball {
  animation: bounce 1s ease-in-out infinite;
}
```

`@keyframes bounce { ... }` defines the named sequence itself — a set of percentage "checkpoints" through the animation's timeline (you can use as many as you like, not just 0/50/100), each describing what the element's styles should be at that exact point. `animation: bounce 1s ease-in-out infinite;` then applies that named sequence to an element: run the `bounce` sequence, taking `1s` total, with an `ease-in-out` timing curve, repeating `infinite` (forever) — other common values instead of `infinite` include a specific number like `3` (run it three times then stop).

`transform` (used above) is worth knowing about even briefly: it lets you move (`translateX/translateY`), resize (`scale`), or rotate (`rotate`) an element visually, without affecting the normal document flow at all (surrounding elements don't shift to compensate) — it's the property you'll reach for constantly alongside both `transition` and `@keyframes`.

CSS custom properties (variables) — define a value once, reuse it everywhere

```
:root {
  --primary-color: steelblue;
  --spacing-unit: 8px;
}

.button {
  background-color: var(--primary-color);
  padding: var(--spacing-unit) calc(var(--spacing-unit) * 2);
}

.link {
  color: var(--primary-color);
}
```

A **custom property** (informally called a "CSS variable") is declared with two leading dashes (`--primary-color`) and read back with `var(--primary-color)`. `:root` is a special selector matching the very top of the document (effectively the same as `html`, but conventionally used specifically for defining

custom properties) — declaring your variables there makes them available for use absolutely anywhere else in your stylesheet.

Why bother, instead of just retyping `steelblue` everywhere you need it? The moment you need to change your site's primary color — a genuinely common, ordinary request — you change it in exactly **one place** (the `:root` declaration), and every single rule referencing `var(--primary-color)` throughout your entire stylesheet updates automatically and consistently. Without custom properties, you'd need to hunt down and manually change every individual occurrence of `steelblue` yourself, risking missing one and ending up with an inconsistent design. This becomes dramatically more valuable as a stylesheet grows past a handful of rules — which is exactly the direction your own projects are headed as this course continues.

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO --> /* TODO */` markers, and check your work against `CHECKLIST.md`.